

HOW TO: Variant2Treatment Web

By Lucía de Lamadrid

The [Variant2Treatment](https://variant2treatment.bio) platform is a web application developed to provide a public interface for allowing users to submit and track variant analyses, together with a protected administration interface for internal order management. The application is accessible through the domain:

<https://variant2treatment.bio>

The platform is divided into three independent services:

- A public frontend developed with **React** and built with **Vite**.
- A separate React-based administration interface for authenticated users.
- A **Node.js** backend responsible for API requests, authentication, database operations, order management, and email notifications.

The three services are containerized and managed through Docker Compose, allowing the application to be built and tested locally in an environment like a production deployment.

1. Public-facing frontend

The frontend directory contains the public-facing React application. This is the website accessed by regular users and includes the main information pages, order-related functionality, and order-tracking interface.

The public frontend is organized around two main pages. **Home.jsx**, located under `src/pages/`, defines the main landing page and assembles the individual content sections stored under `src/sections/`, such as Hero, Information, Publication, Steps, Team, and Technology. Each section is implemented as an independent React component with its corresponding JSX and CSS files. The `.jsx` files contain the React component responsible for the structure and behavior of that section, while the `.css` files contain the visual styling associated with each component.

App.jsx defines the overall application structure and routing between the home page and the order-tracking page. The tracking functionality is implemented in the TrackOrder section and is accessed through the navigation bar; users enter a tracking ID to retrieve the status and information associated with a specific order.

The `components/` folder contains the public website navigation bar (*Navbar*) and the footer displayed across the website (*Footer*).

main.jsx is the application entry point and mounts the main App component into the HTML document, while **index.css** contains global styles applied across the entire frontend, including general typography, page layout, and shared visual settings.

The `src/assets/` folder stores images, icons, and other static files. To use a new image, add it to this folder, import it in **assets.js**, and assign it a descriptive name in the exported assets object. The image can then be used in any JSX component by importing assets and referencing it as `assets.imageName`.

Variant to Treatment

TechnologyTeamHow it worksTrack your order

Navbar

From Variant Discovery to Clinical Action

Functional interpretation of genetic variants to accelerate precision treatment decisions.

Explore our technologySubmit variant

Pilot implementation funded by BETA-HEALTH.

Hero

About Us

We provide functional classification of genetic variants using CRISPR-based precision genome editing and MiniGene splice assays. The service is a part of the MDxCore unit at the Department of Clinical Biochemistry, Rigshospitalet, Copenhagen University Hospital.

Our platform provides experimental evidence to support clinical variant interpretation and is currently available as a pilot initiative funded by BETA-HEALTH entitled "Variant to Treatment."

Our Technology

CRISPR AssayMiniGene Assay

Functional Variant Interpretation Using CRISPR-based Assays

Our platform employs CRISPR-based genome editing and sequencing-based readout to enable direct functional assessment of genetic variants within their endogenous genomic context.

By introducing defined variants into human cells and measuring their biological consequences, our approach provides quantitative functional evidence that supports variant classification framework.

Gene Coverage

Currently Available

BRC A1BRC A2

Genes in Development

ATMPTEN

Additional clinically relevant genes will be incorporated following validation.

Technology

Publication

DISCLAIMER

Functional assay results provide experimental evidence to support variant interpretation and should be integrated with clinical, genetic, and familial information in accordance with ACMG guidelines.

Final variant classification remains the responsibility of the requester.

Meet the Team

Maria Rossing
Chief Physician

Muthiah Bose
Academic Researcher

Enrique Gómez
Research Assistant

Sofie Eriksen
Research Technician

Team

Submit your variant here

Requests are only accepted when submitted by authorized professionals or medical institutions.

Variant Information

Type of assay:

☒ VUS classification☐ MiniGene Assay

Gene:

--Select--

Variant:

e.g. c.3A>G (p.MetVal)

Any additional comments?

Your Personal Details

Full Name:

Institution:

EAN Number:

Email:

Phone Number:

City:

ZIP Code:

Submit

How it works

1 Submit Variant Information

Complete the submission form with:

- Gene name
- MANE transcript reference
- Codon-level variant details
- Genome build hg38
- Contact information

Accurate transcript and codon information are essential for assay design.
No biological specimen is required.

2 Assay Design

Our team designs the required reagents.

3 Functional Assessment

Variants are experimentally evaluated in controlled cellular systems to determine functional impact.

Turnaround Time

- Variant classification assay: 6-8 weeks
- MiniGene assay: Approximately 3 months

4 Track Progress

Upon submission, a tracking ID is issued. Progress can be monitored via the "Track Your Order" function on the website.

5 Reporting

A final report summarizing functional results will be sent by email to requester and subsequently uploaded in ClinVar.

Steps

Information

MDxCore Department

Rigshospitalet

Copenhagen, Denmark

Contact Us

variant@treatchment.rigshospitalet.rigshospitalet.dk

+45 35 45 87 43

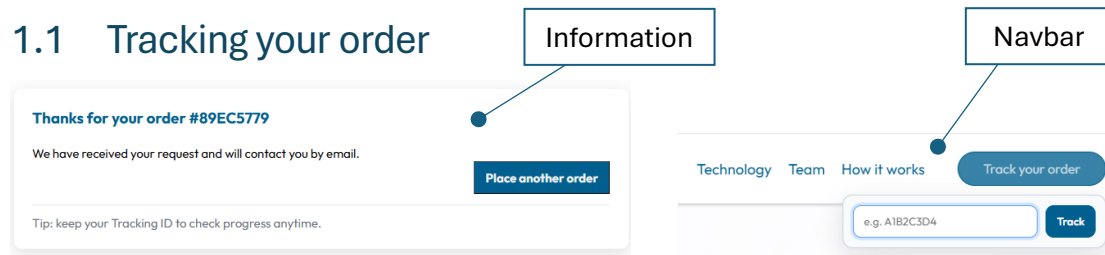
Variant to Treatment

The service is currently being rolled out as a pilot model, funded by BETA-HEALTH under the project name: "Variant to Treatment".

Footer

The website has been implemented to adapt correctly to different screen sizes, including desktop, iPad, and mobile devices. These responsive layouts can be modified in the CSS files using the `@media` rules.

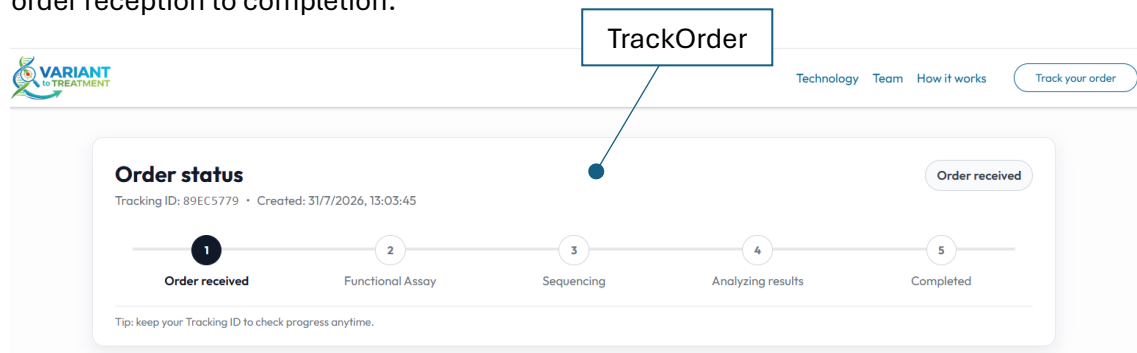
1.1 Tracking your order



Confirmation message displayed in place of the submission form after the request has been submitted, including the assigned tracking ID

Tracking menu in the Navbar, where users can enter their tracking ID.

Order-tracking page showing the current status and the different processing steps, from order reception to completion:

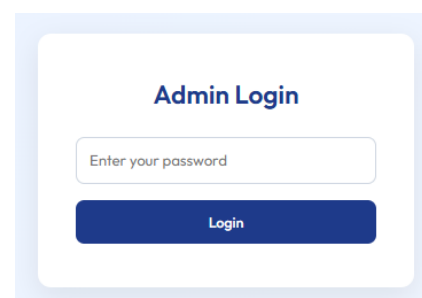


2. Administration interface

The admin authentication interface is handled by:

- **Login.jsx:** displays the password form and sends the entered password to the backend login endpoint; when authentication is successful, it stores the returned JWT as adminToken in localStorage and redirects the user to the admin panel.
- **ProtectedRoute.jsx:** protects the private admin routes by checking that this token exists, can be decoded, and has not expired, redirecting unauthenticated users to the login page.
- **Login.css:** contains the visual styling for the login screen, including the form, input field, button, and error message.

The actual administrator password is securely defined in the backend `.env` file. The established password now is `variant_admin_2026`.





The **Orders.jsx** component connects the admin panel to the backend. It retrieves all submitted orders from the `/api/orders/list` endpoint and displays them in a paginated table, allowing laboratory staff to review the request details, customer information, submission date, comments, and current progress.

Using the **Sidebar**, the table can be **filtered** by assay, gene, status, or keyword, while the available assay and gene options are also obtained from the backend. Laboratory staff can update an order directly from the table using the status dropdown; the new status is sent to `/api/orders/status`, and the *TrackOrder* page will be updated.

3. Backend

This is the part of the application that connects the website to the database and manages the main server-side operations. It is built with Node.js and Express, uses MongoDB to store the orders, and exposes API endpoints that the frontend calls using Axios.

When a user submits a request, the backend validates the information, creates a unique tracking ID, stores the order in MongoDB, and sends a confirmation email.

- **config/db.js:** establishes the connection between the backend and the MongoDB database using Mongoose. The database address is obtained from `MONGODB_URI` in the `.env` file.
- **Controllers/orderController.js:** contains the main logic used to manage orders.
 - *PlaceOrders:* creates new orders, generates their tracking IDs, saves them in the database, sends confirmation emails.
 - *ListOrders:* retrieves the orders for the admin panel, applies filters and pagination.
 - *updateStatus:* updates orders status.
 - *getAssays, getGenes:* provides the information required for the gene and assay filters.
 - *trackOrder:* allows users to retrieve the status of an order using its trackID.
- **Emails/OrderConfirmation.jsx:** defines the design and content of the confirmation email sent after an order is submitted. The URLs used in the email are created from `FRONTEND_URL`, which is defined in the backend `.env` file.

For email delivery, the application uses Resend. The domain must be verified in the Resend dashboard by adding the required DNS records to the domain configuration. The backend sends emails from an address such as noreply@variant2treatment.bio. The Resend API key must not be exposed in the frontend or committed to the repository.

- **Middleware/auth.js:** contains the backend authentication middleware for protected administrator operations.
- **Models/orderModel.js:** defines the structure of an order in MongoDB. Each order contains a unique tracking ID, the selected assay, gene and variant, the customer's contact and billing information, optional comments, and the current order status.
- **Routes/loginRoute.js:** defines the administrator login endpoint. It compares the password received from the frontend with `ADMIN_PASSWORD` from the backend `.env` file.
- **Routes/orderRoute.js:** connects each order-related API URL to the corresponding function in `orderController.js` (e.g. `GET /api/orders/list` → `listOrders()`).
- **Services/emailService.js:** responsible for sending the confirmation email through the Resend service. It uses `RESEND_API_KEY` from the `.env` file and inserts the customer's information into the `OrderConfirmationEmail`.
- **Server.js:** the main entry point of the backend. It loads the environment variables, creates the Express application, enables JSON requests, configures CORS so that the frontend can communicate with the backend, connects to MongoDB, and activates the login and order routes. Finally, it starts the server using the port defined in `.env`.

4. Development VS Production

The application uses separate configurations for development and production.

Each part of the application has separate environment configuration files for **development** and **production**. Sensitive information, such as the MongoDB connection string, admin password, JWT secret, and Resend API key, must only be stored in the backend environment files and should never be committed to Git.

The *development environment* contains values used when running the application locally, such as localhost API URLs, while the *production environment* contains the URLs and configuration required by the deployed application. This allows the same source code to be used in both environments without manually changing URLs or configuration values.

Environment variables used in the React applications (typically prefixed with `VITE_`) should not contain secrets, as they are included in the client-side build and can be accessed by the browser.

In **development**, `docker-compose.yml` exposes the frontend, admin and backend ports directly (5173, 5174 and 4000), allowing the Vite applications to communicate with the backend through <http://localhost:4000>.

For local development, run the application using: `docker compose up --build`.

In **production**, `docker-compose.prod.yml` uses internal Docker networking and expose instead of publishing the backend port directly.

For production deployment on the server, first update the local copy of the repository and make sure the required `.env.production` files and production secrets are present on the server.

5. Email Automation w/ Resend

The application uses **Resend** to automatically send two emails whenever a new variant submission is successfully created:

1. A **confirmation email** to the person who submitted the variant, including the tracking ID and a link to check its status.
2. An **internal notification email** to the laboratory, informing the team that a new variant submission has been received.

Resend Configuration

To enable email delivery, create an account in Resend and generate an **API key**. This key must be added to the **backend environment files** for both development and production:

- `RESEND_API_KEY=re_XXXXXXXXX`

The API key is **confidential** and must never be committed to GitHub. The files were shared through email to Mauricio Moldes.

Domain Verification

Before Resend can send emails from the application domain, *variant2treatment.bio* must be verified in Resend. Resend provides a set of **DNS records** that must be added to the domain configuration in **one.com**. Once the DNS records have propagated and the domain appears as verified in Resend, the application can use:

noreply@variant2treatment.bio

as the sender address.

Implementation

The Resend configuration and email-sending functions are in: *backend/services/emailService.js*

Two main functions are used:

- `sendOrderConfirmation()`: Sends the confirmation email to the person who submitted the variant.
- `sendNewSubmissionNotification()`: Sends an internal notification to the lab.

The emails are triggered from *orderController.js* after the new submission has been saved to MongoDB. This ensures that email notifications are only generated for successfully stored submissions.

6. MongoDB Configuration

The application uses **MongoDB** to store all variant submissions, customer information, tracking IDs, and submission status information.

The backend connects to the database using the MongoDB connection string stored in the environment variables:

- `MONGODB_URI=mongodb+srv://...`

This must be configured in both the development and production **backend env files**.

MongoDB Atlas Configuration

If MongoDB Atlas is used, the following components are required:

- A MongoDB Atlas account.
- A MongoDB project and cluster.
- A database user with the required permissions.
- Network access configured to allow the application server to connect.
- A valid MongoDB connection string.

The database username and password used inside `MONGODB_URI` correspond to a **MongoDB database user**, which is different from the user account used to log in to MongoDB Atlas.

The connection string should be stored only in the environment files and must never be committed to GitHub.

MongoDB Changes

There are three different situations to consider:

1. Changing the MongoDB Atlas Account

If another person needs to manage the existing MongoDB project, simply add their MongoDB Atlas account to the current project or organization.

The database and cluster remain the same, so:

- No data migration is needed.
- `MONGODB_URI` does not need to change.

2. Changing the Database User

The Database User is the username and password used by the application to connect to MongoDB. If a new Database User is created, update `MONGODB_URI` with the new username and password in both development and production environments.

`MONGODB_URI=mongodb+srv://NEW_USER:NEW_PASSWORD@...`

3. Moving to a New Database or Cluster

If the application is moved to a completely new MongoDB cluster, the existing data will not automatically be available there.

In this case:

1. Create the new cluster.
2. Create a Database User.
3. Configure network access.
4. Migrate the existing database if the previous submissions need to be preserved.
5. Replace MONGODB_URI with the connection string of the new cluster.
6. Restart the backend and verify that the application can read and store submissions correctly.

In short: changing the Atlas account only changes who manages the database, changing the Database User requires updating MONGODB_URI, and changing the cluster requires both updating MONGODB_URI and potentially migrating the existing data.